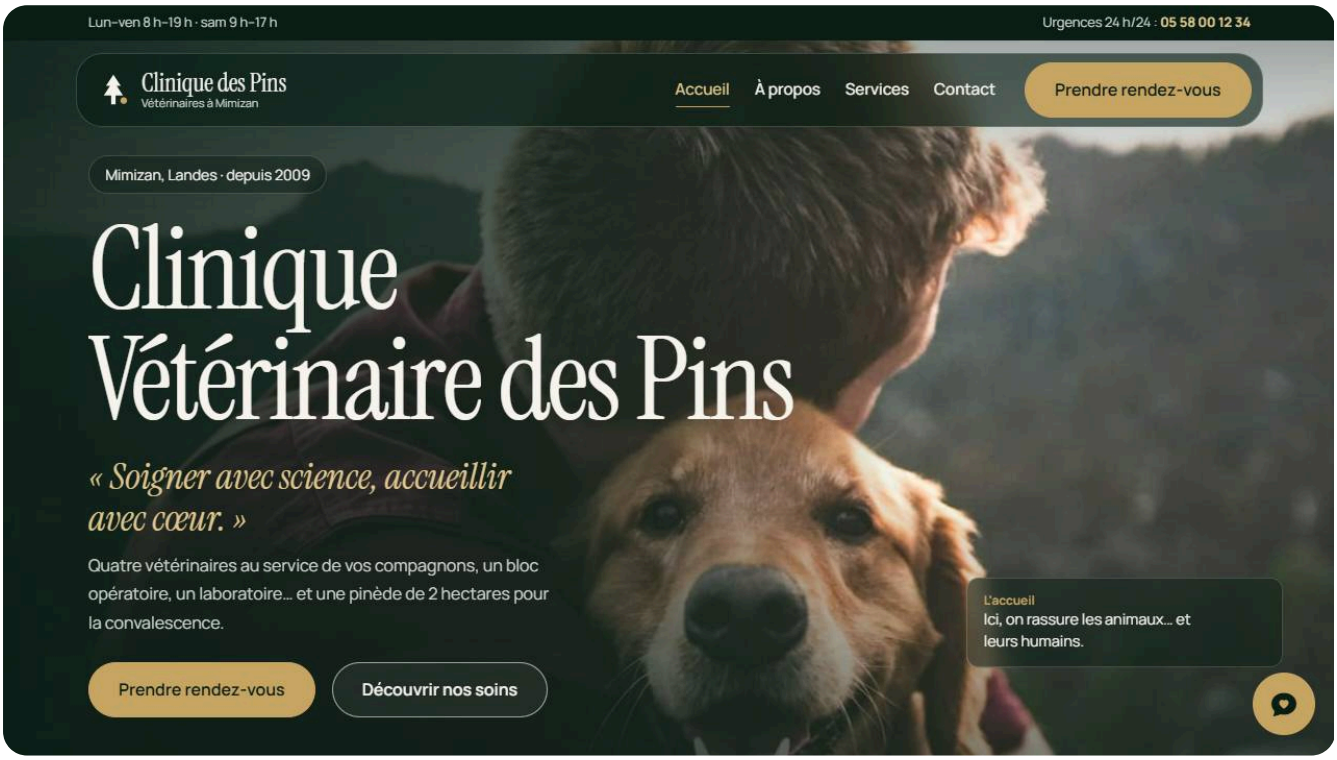
Guide technique · développeurs juniors

Comment fonctionne le site de la Clinique Vétérinaire des Pins

Architecture, code, sécurité, tests et déploiement, expliqués pas à pas à partir du vrai code du projet.



Site en ligne	clinique-veterinaire-des-pins.vercel.app
Code source	github.com/Max-De-Jesus/clinique-veterinaire-des-pins
Date du guide	1er novembre 2026

Sommaire

1. Le projet en cinq minutes
2. Démarrer sur votre machine
3. Le HTML : quatre pages statiques
4. Le CSS : variables, mise en page, responsive
5. Le JavaScript et les animations
6. Le formulaire : du navigateur à Gmail
7. Le chatbot Sylva
8. La sécurité
9. Les tests et les preuves
10. Le déploiement
11. Comment nous avons obtenu HTTPS
12. Les erreurs réelles et leurs leçons
13. Recettes pour modifier le site
14. Exercices
15. Limites connues et glossaire

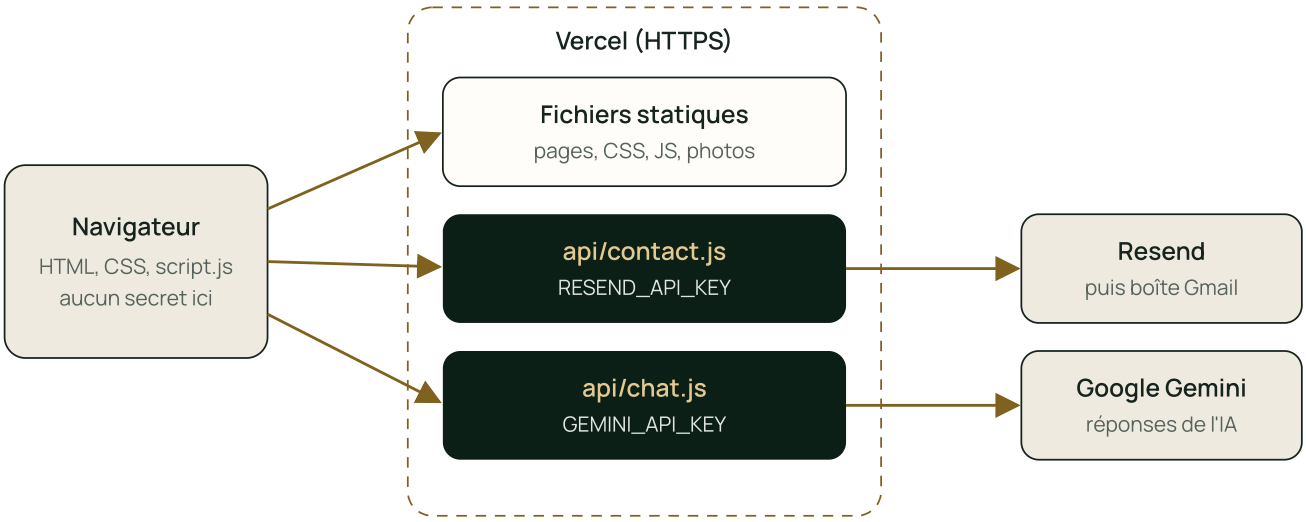
Comment lire ce guide. Si vous débutez, lisez les chapitres 1 à 6 dans l'ordre. Ouvrez ensuite les fichiers cités en parallèle : les extraits sont copiés du code réel au moment où ce PDF a été généré. Si le code a évolué depuis, c'est le code qui fait foi.

1. Le projet en cinq minutes

C'est le site vitrine d'une clinique vétérinaire **fictive** à Mimizan, dans les Landes, réalisé pour un projet étudiant « Hackathon XHTML & CSS ». Le sujet impose quatre pages, du HTML valide, Bootstrap, un CSS et un JavaScript dans des fichiers séparés, et un hébergement gratuit sur Vercel. Deux fonctions dépassent le simple site vitrine : un formulaire qui envoie un vrai e-mail, et un chatbot connecté à une IA.

L'idée centrale : un site statique et deux petites fonctions serveur

Il n'y a ni framework, ni base de données, ni étape de compilation. Les pages sont des fichiers HTML servis tels quels. Le seul code serveur tient dans le dossier `api/`, et il n'existe que pour une raison : **protéger des clés secrètes**. Tout ce qui s'exécute dans un navigateur est lisible par n'importe quel visiteur, donc une clé API ne peut pas s'y trouver.



Le navigateur ne parle qu'à Vercel, sur le même domaine. Seules les fonctions serveur connaissent les clés et contactent les services externes.

Qui fait quoi

Élément	Rôle
`index.html`, `a-propos.html`, `services.html`, `contact.html`	Les quatre pages du site
`css/style.css`	Toute l'apparence : couleurs, mise en page, animations
`js/script.js`	Le comportement dans le navigateur
`api/contact.js`	Valide le formulaire et envoie l'e-mail
`api/chat.js`, `api/_contexte.js`	Interroge l'IA avec le contexte de la clinique
`api/_commun.js`	Outils partagés (réponse JSON, limite de débit)
`assets/`	45 photos Unsplash et le favicon

`tests/`, `tools/`	Tests automatiques, serveur local, lanceur de tests
`docs/`	Décisions d'architecture (ADR), passation, ce guide
`vercel.json`, `.vercelignore`	En-têtes de sécurité, fichiers exclus du déploiement

Astuce. Les fichiers de `api/` dont le nom commence par `_` ne deviennent pas des adresses web : Vercel les ignore comme routes. C'est pour cela que `_commun.js` renvoie une erreur 404 si on essaie de l'ouvrir.

2. Démarrer sur votre machine

Il vous faut **Node.js 20 ou plus** et Git. Le projet n'a aucune dépendance à installer : pas de `npm install`.

```
git clone https://github.com/Max-De-Jesus/clinique-veterinaire-des-pins.git
cd clinique-veterinaire-des-pins
npm run dev          # site sur http://localhost:3000
```

Commande	Ce qu'elle fait
`npm run dev`	Lance le serveur local (`tools/dev-server.mjs`), qui imite Vercel
`npm test`	Lance tous les tests et vérifie leur nombre
`npm run validate`	Contrôle le HTML des 4 pages avec html-validate
`npm run images`	Retélécharge les photos (voir l'avertissement du chapitre 13)

Et les clés ?

Sans clé, le site fonctionne, mais le chatbot et le formulaire répondent « indisponible » (HTTP 503). C'est voulu : il n'y a jamais de faux succès. Pour les tester en local, définissez les variables dans votre terminal :

```
# PowerShell : remplacez par vos propres clés, jamais dans un fichier versionné
$env:GEMINI_API_KEY="votre-cle"
$env:RESEND_API_KEY="votre-cle"
$env:CONTACT_DESTINATAIRE="vous@example.com"
npm run dev
```

Règle absolue. Une clé ne va jamais dans le code, ni dans Git, ni dans un message de discussion. Le fichier `.env.example` ne contient que des valeurs fictives : c'est un modèle, pas un endroit où ranger un secret.

3. Le HTML : quatre pages statiques

Chaque page suit le même squelette : l'en-tête, le contenu principal, le pied de page, puis les scripts. Les balises **sémantiques** (`<header>`, `<nav>`, `<main>`, `<section>`, `<footer>`) disent à quoi sert chaque zone, ce qui aide les lecteurs d'écran et le référencement.

Une carte de service, exemple type

Les classes `card`, `col-md-6` et `col-lg-4` viennent de Bootstrap. Les classes en français comme `carte-service` et `apparition` sont les nôtres. L'attribut `data-categorie` sert aux filtres en JavaScript.

services.html

```
<!-- Carte 1 -->
<div class="col-md-6 col-lg-4 apparition" data-categorie="soins">
  <article class="card carte-service">
    <div class="cadre-image">
      <img src="assets/img/examen-teckel.jpg" alt="Vétérinaire examinant un teckel" width="900"
height="1350" loading="lazy">
    </div>
    <div class="card-body">
      <span class="etiquette">Soins courants</span>
      <h3 class="card-title">Consultations</h3>
      <p class="card-text">Examen complet de 30 minutes, conseils nutrition et comportement, suivi des
maladies chroniques.</p>
      <p class="prix">À partir de 45 €</p>
    </div>
  </article>
</div>
<!-- Carte 2 -->
<div class="col-md-6 col-lg-4 apparition" data-categorie="soins">
  <article class="card carte-service">
    <div class="cadre-image">
      <img src="assets/img/vaccin-chien.jpg" alt="Injection d'un vaccin à un petit chien" width="900"
height="1350" loading="lazy">
    </div>
    <div class="card-body">
      <span class="etiquette">Soins courants</span>
      <h3 class="card-title">Vaccins &amp;amp; prévention</h3>
      <p class="card-text">Primo-vaccination, rappels, antiparasitaires, identification par puce et
passeport européen.</p>
      <p class="prix">Rappel vaccinal : 62 €</p>
    </div>
  </article>
</div>
```

La grille Bootstrap en deux phrases

Une rangée (`row`) compte 12 colonnes. `col-md-6 col-lg-4` veut dire : 6 colonnes sur 12 (la moitié) à partir d'une tablette, 4 sur 12 (un tiers) à partir d'un grand écran, et toute la largeur sur téléphone. Sur la page Services, certaines cartes font `col-lg-8` : un test vérifie que le total reste un multiple de 12, pour qu'aucune rangée ne reste incomplète.

Les images

Chaque image porte un `alt` qui la décrit, ses vraies dimensions et `loading="lazy"` (sauf celle du haut de l'accueil). Les dimensions permettent au navigateur de réserver la place avant le chargement, pour que le texte ne saute pas. `lazy` retarde le téléchargement des images hors de l'écran, ce qui accélère la page.

Piège de ce projet. L'en-tête et le pied de page sont **copiés dans les 4 fichiers**, car le sujet impose du HTML statique sans gabarit. Toute modification doit être reportée dans les 4 pages.

4. Le CSS : variables, mise en page, responsive

Tout le style personnalisé est dans `css/style.css`, découpé en 19 sections annoncées par un sommaire en haut du fichier. Le sujet interdit tout style en ligne : pas d'attribut `style=""` ni de balise `<style>`, et un test le vérifie.

Les variables : changer une couleur à un seul endroit

Les couleurs ne sont jamais écrites en dur dans les composants. On les définit une fois dans `:root`, puis on les utilise avec `var(--or)`. Pour changer l'accent de tout le site, il suffit de modifier une ligne.

css/style.css

```
/* Fonds */
--nuit: #0c1f17;
--nuit-claire: #15301f;
--ivoire: #f8f5ef;
--ivoire-fonce: #efeadf;
--blanc: #fffdf9;
/* Texte */
--encre: #16241d;
--encre-douce: #56635c;
--texte-sur-nuit: #cfd8d0;
/* Un seul accent : l'or (version foncée pour le texte sur fond clair) */
--or: #c8a664;
--or-clair: #e2c98f;
--or-fonce: #80621f;
```

Notez les deux ors. `--or` est lisible sur fond sombre, mais pas assez contrasté sur fond clair : pour du texte sur ivoire, on utilise `--or-fonce`. Le contraste n'est pas un détail de goût, c'est ce qui rend le texte lisible pour tous.

Grille et Flexbox

Bootstrap gère les colonnes principales. Pour les mises en page sur mesure, le CSS utilise **Grid** en deux dimensions, comme la mosaïque de la galerie où certaines photos occupent deux colonnes et deux lignes :

css/style.css

```
.galerie {
  display: grid;
  grid-template-columns: repeat(4, minmax(0, 1fr));
  grid-auto-rows: clamp(150px, 16vw, 250px);
  gap: 0.75rem;
}
```

Le responsive, du grand au petit

Les *media queries* en fin de fichier adaptent la page à la largeur de l'écran. Le projet utilise aussi `clamp(min, idéal, max)`, qui fait varier une taille en continu sans point de rupture :

```
font-size: clamp(3.5rem, 1.5rem + 6.8vw, 7.75rem);
/* jamais moins de 3.5rem, jamais plus de 7.75rem, et proportionnel à l'écran entre les deux */
```

Un état caché seulement si JavaScript fonctionne

Les éléments qui apparaissent au défilement partent invisibles. Mais si JavaScript ne se charge pas, ils ne doivent pas rester cachés. D'où cette astuce : le script ajoute la classe `js` sur `<html>`, et le CSS ne cache que dans ce cas.

css/style.css

```
.js .apparition {
  opacity: 0;
  transform: translateY(36px);
  transition: opacity 0.9s var(--courbe), transform 0.9s var(--courbe);
}

.js .apparition.visible {
  opacity: 1;
  transform: none;
```

Respecter ceux qui demandent moins de mouvement

Certaines personnes sont gênées par les animations et le signalent dans les réglages de leur système. Le site le détecte et coupe tout mouvement :

css/style.css

```
@media (prefers-reduced-motion: reduce) {
  *,
  *::before,
  *::after {
    animation: none !important;
    transition: none !important;
    scroll-behavior: auto !important;
  }

  .js .apparition {
    opacity: 1;
    transform: none;
  }

  .js .apparition:not(.visible) .cadre-image img,
  .js .apparition:not(.visible) .photo img {
    transform: none;
  }

  .js .entete-section.apparition:not(.visible)::before {
    transform: none;
  }

  [data-parallaxe] {
    transform: none;
  }
}
```

Pourquoi animer seulement transform et opacity

Déplacer un élément avec `top` ou `margin` oblige le navigateur à recalculer toute la mise en page à chaque image, ce qui saccade. `transform` et `opacity` sont traités à part par la carte graphique et restent fluides : presque toutes les animations du site utilisent ces deux propriétés.

5. Le JavaScript et les animations

Le script lance une fonction par fonctionnalité dès que la page est prête. C'est le premier endroit à lire dans `js/script.js` :

js/script.js

```
document.addEventListener('DOMContentLoaded', () => {
  initialiserNavigation();
  initialiserCarrousel();
  initialiserTitresAnimes();
  initialiserProgressionEtParallaxe();
  initialiserApparitions();
  initialiserCompteurs();
  initialiserGalerie();
  initialiserFiltres();
  initialiserFormulaireContact();
  afficherStatutOuverture();
  initialiserChatbot();
});
```

Chaque fonction commence par vérifier que ses éléments existent dans la page : `if (!formulaire) return;`. Le même fichier est chargé sur les 4 pages, mais le formulaire n'existe que sur l'une d'elles.

Faire apparaître les éléments au défilement

Écouter l'événement `scroll` et mesurer chaque élément serait coûteux. `IntersectionObserver` demande au navigateur de nous prévenir quand un élément entre à l'écran. On ajoute alors la classe `visible`, et le CSS fait la transition. `unobserve` arrête la surveillance : l'animation ne joue qu'une fois.

js/script.js

```
const observateur = new IntersectionObserver((entrees) => {
  entrees.forEach((entree) => {
    if (entree.isIntersecting) {
      entree.target.classList.add('visible');
      observateur.unobserve(entree.target); // une seule animation par élément
    }
  });
}, { threshold: 0.12, rootMargin: '0px 0px -40px 0px' });

elements.forEach((el) => observateur.observe(el));
```

Révéler un titre mot à mot

Le HTML contient le titre en entier. Le script le découpe ensuite en mots, chacun dans deux `<span>` : le cadre extérieur masque ce qui dépasse, le mot intérieur monte depuis le bas. La variable CSS `--rang` décale chaque mot. Le texte reste du vrai texte, lisible par les lecteurs d'écran et les moteurs de recherche.

js/script.js

```
function initialiserTitresAnimes() {
  if (reduireAnimations) return;

  document.querySelectorAll('.titre-anime').forEach((titre) => {
    const mots = titre.textContent.split(' ');
    titre.textContent = '';
    mots.forEach((mot, rang) => {
      const cadre = document.createElement('span');
      const interieur = document.createElement('span');
      cadre.className = 'mot';
      interieur.className = 'mot-interieur';
      interieur.textContent = mot;
      interieur.style.setProperty('--rang', String(rang));
      cadre.appendChild(interieur);
      titre.append(cadre, rang < mots.length - 1 ? ' ' : '');
    });
  });
}
```

css/style.css

```
.mot-interieur {
  display: inline-block;
  animation: motMonte 1.1s var(--courbe) both;
  animation-delay: calc(var(--rang, 0) * 75ms + 150ms);
}
```

La parallaxe, sans ralentir la page

Un événement `scroll` peut se déclencher bien plus souvent que l'écran ne se redessine. `requestAnimationFrame` regroupe le calcul : un seul par image affichée. Le décalage est aussi plafonné à 40 px, pour qu'une photo ne s'écarte jamais trop de sa place. Et sur téléphone, il n'y a pas de parallaxe du tout.

js/script.js

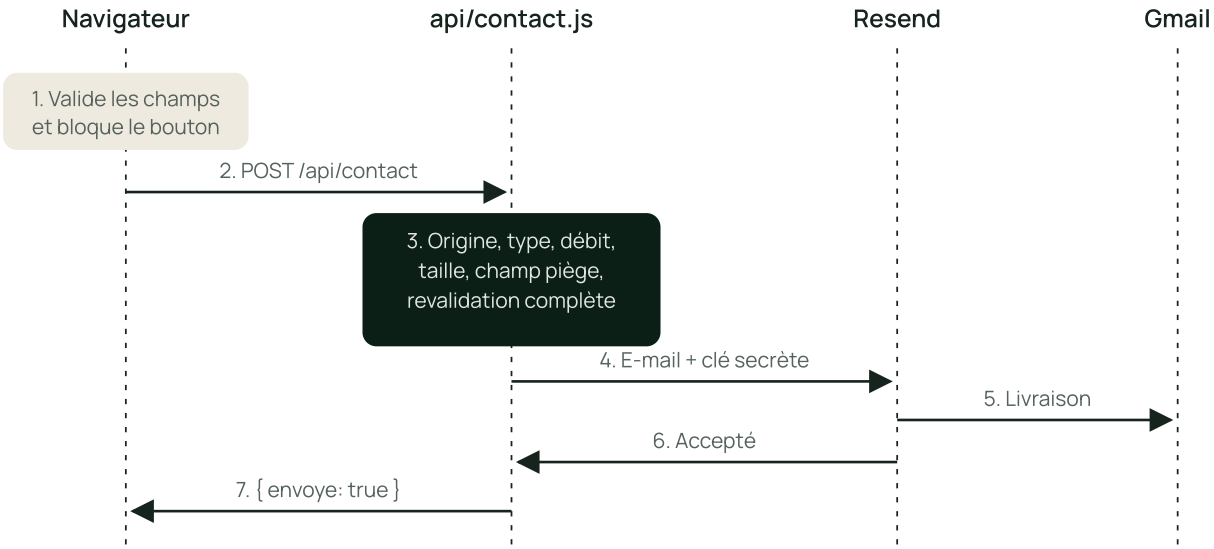
```
const demanderCalcul = () => {
  if (calculPrevu) return;
  calculPrevu = true;
  requestAnimationFrame(mettreAJour);
};
```

Le carrousel

Il vient de Bootstrap et fonctionne avec de simples attributs HTML (`data-bs-ride`, `data-bs-slide`). Notre script n'ajoute que deux choses : il coupe le défilement automatique si le visiteur demande moins de mouvement, et il précharge les photos suivantes pour que le fondu ne montre jamais de vide.

6. Le formulaire : du navigateur à Gmail

Un site statique ne peut pas envoyer d'e-mail seul : il faudrait placer une clé dans le navigateur, où tout le monde pourrait la lire. Le navigateur appelle donc notre fonction serveur, qui détient la clé et appelle Resend, un service d'envoi d'e-mails.



Le succès n'est annoncé au visiteur qu'à l'étape 7, une fois que Resend a réellement accepté l'envoi.

Côté navigateur

La validation dans le navigateur sert au **confort** : l'erreur s'affiche tout de suite. Elle ne protège rien, car n'importe qui peut envoyer une requête sans passer par la page. En cas d'échec, le formulaire reste rempli : le visiteur ne perd pas ce qu'il a écrit.

js/script.js

```
let reponse;
try {
  reponse = await fetch(CONTACT.point_api, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(champs)
  });
} catch (erreurReseau) {
  throw new Error(CONTACT.erreur_reseau);
}

const donnees = await reponse.json().catch(() => ({}));
if (!reponse.ok || donnees.envoye !== true) {
  throw new Error(donnees.erreur || CONTACT.erreur_inconnue);
}
```

Côté serveur : ne jamais faire confiance

Le serveur revalide tout. Remarquez l'ordre des contrôles : on refuse d'abord ce qui ne coûte rien à refuser, avant de lire le corps et d'appeler un service externe.

api/contact.js

```
export async function POST(request) {
  const cleApi = process.env.RESEND_API_KEY;
  const destinataire = process.env.CONTACT_DESTINATAIRE;
  if (!cleApi || !destinataire) {
    console.error('[contact] RESEND_API_KEY ou CONTACT_DESTINATAIRE absente de l\'environnement');
    return json(503, { erreur: `L'envoi de messages n'est pas encore activé. Appelez-nous au
${TELEPHONE_SECOURS}.` });
  }

  if (!origineAutorisee(request)) return json(403, { erreur: 'Origine non autorisée.' });
  if (!(request.headers.get('content-type') || '').toLowerCase().startsWith('application/json')) {
    return json(415, { erreur: 'Format de requête non pris en charge.' });
  }
  if (depasseLimite(adresseIP(request))) {
    return json(429, { erreur: `Vous avez envoyé plusieurs messages : patientez quelques minutes ou
appelez le ${TELEPHONE_SECOURS}.` });
  }

  // La taille annoncée évite de lire un corps trop gros ; la taille réelle reste vérifiée ensuite
  if (Number(request.headers.get('content-length')) > TAILLE_CORPS_MAX) {
    return json(413, { erreur: 'Message trop volumineux.' });
  }
  const brut = await request.text();
  if (Buffer.byteLength(brut) > TAILLE_CORPS_MAX) return json(413, { erreur: 'Message trop volumineux.'
});

  let corps;
  try {
    corps = JSON.parse(brut);
  } catch {
    return json(400, { erreur: 'Requête illisible.' });
  }

  // Champ piège invisible : seul un robot le remplit. Il croit avoir réussi, rien n'est envoyé.
  if (corps && typeof corps === 'object' && texte(corps.confirmation_interne)) return json(200, { envoye:
true });

  const resultat = validerSoumission(corps);
```

Les protections, une par une

Protection	Contre quoi
Destinataire côté serveur	Le visiteur ne peut pas choisir à qui écrire : le site ne peut pas servir de relais de spam vers des tiers.
Champ piège (`confirmation_interne`)	Invisible pour un humain. Un robot qui le remplit reçoit un faux succès, et rien n'est envoyé.
Limite de débit	5 envois par 10 minutes et par adresse IP, pour éviter d'inonder la boîte et d'épuiser le quota gratuit.
Contrôle d'origine	Empêche le navigateur d'un autre site d'envoyer le formulaire. Un script comme `curl` peut l'éviter : la vraie barrière reste la limite de débit.
Taille limitée à 8 Ko	Refuse les requêtes énormes qui occupent la mémoire.

Une seule adresse possible	Sans cela, « Répondre » depuis Gmail pourrait viser des destinataires choisis par un attaquant.
E-mail en texte brut	Pas de HTML, donc pas de code malveillant à échapper.
Erreurs masquées	Le visiteur ne voit jamais les détails internes, et les journaux ne contiennent pas ses données.

L'e-mail envoyé

`reply_to` contient l'adresse du visiteur : dans Gmail, le bouton « Répondre » lui écrit directement. Le destinataire vient uniquement de la variable d'environnement.

api/contact.js

```
export function composerEmail(donnees, destinataire, expediteur) {
  const libelleAnimal = ANIMAUX[donnees.animal];
  return {
    from: expediteur,
    to: [destinataire],
    reply_to: donnees.email,
    subject: `[Clinique des Pins] Message de ${donnees.nom} (${libelleAnimal})`,
    text: [
      'Nouveau message reçu depuis le formulaire de contact du site.',
      '',
      `Nom : ${donnees.nom}`,
      `E-mail : ${donnees.email}`,
      `Téléphone : ${donnees.telephone || 'non renseigné'}`,
      `Animal : ${libelleAnimal}`,
      '',
      'Message :',
      donnees.message,
      '',
      '— Pour répondre au visiteur, utilisez simplement « Répondre ».'
    ].join('\n')
  };
}
```

Limite à connaître. Sans nom de domaine vérifié, Resend n'envoie qu'à l'adresse de son propre compte, et l'expéditeur affiché est `onboarding@resend.dev`. Ces choix sont expliqués dans `docs/adr/0002-envoi-formulaire-par-resend.md`.

7. Le chatbot Sylva

Le mécanisme ressemble à celui du formulaire : le navigateur appelle `/api/chat`, qui ajoute la clé et interroge Gemini. Trois idées sont à retenir.

1. L'IA n'a pas de mémoire : on lui renvoie la conversation

À chaque question, le navigateur envoie les 12 derniers messages. La conversation est gardée dans `sessionStorage`, qui s'efface à la fermeture de l'onglet. Le serveur vérifie chaque message : rôle autorisé, 500 caractères au maximum.

api/chat.js

```
export function convertirHistorique(messages) {
  if (!Array.isArray(messages) || messages.length === 0 || messages.length > MESSAGES_MAX) return null;

  const contenus = [];
  for (const message of messages) {
    if (!message || typeof message.texte !== 'string') return null;
    if (message.role !== 'visiteur' && message.role !== 'assistant') return null;
    const texte = message.texte.trim().slice(0, LONGUEUR_MESSAGE_MAX);
    if (!texte) return null;
    contenus.push({ role: message.role === 'visiteur' ? 'user' : 'model', parts: [{ text: texte }] });
  }

  // Gemini exige que la conversation commence et se termine par l'utilisateur
  while (contenus.length > 0 && contenus[0].role !== 'user') contenus.shift();
  if (contenus.length === 0 || contenus[contenus.length - 1].role !== 'user') return null;
  return contenus;
}
```

2. Le « contexte » fait toute la qualité des réponses

`api/_contexte.js` contient un long texte envoyé à chaque appel : l'identité de Sylva, son ton, les horaires, les tarifs et surtout ses règles. Sans lui, Gemini ne saurait rien de la clinique. C'est lui qui impose de ne jamais poser de diagnostic, de renvoyer vers le téléphone en cas d'urgence et de ne rien inventer.

api/_contexte.js

```
## Règles impératives
- Tu ne poses jamais de diagnostic et ne prescris jamais de médicament ni de dosage : tu orientes vers une consultation.
- Si la situation évoque une urgence (intoxication, saignement, difficulté à respirer, convulsions, ventre gonflé, traumatisme, mise bas difficile…) : dis immédiatement d'appeler le 05 58 00 12 34.
- Tu ne réponds qu'aux sujets liés à la clinique et à la santé ou au bien-être animal. Pour tout autre sujet, recentre poliment.
- N'invente aucune information absente de ce contexte (prix, nom, service) : propose plutôt d'appeler la clinique.
```

3. Prévoir la panne du service externe

Gemini répond parfois 503 (surcharge) ou 429 (quota : l'offre gratuite limite à 5 requêtes par minute et par modèle). Dans ce cas, et seulement dans ce cas, la fonction réessaie **une seule fois** sur un autre modèle. Une clé refusée ne déclenche aucun nouvel essai : le résultat serait le même.

api/chat.js

```
let { reponse, donnees } = await appelerGemini(modele, cleApi, contenus);

// Un seul nouvel essai, et seulement pour une surcharge : une clé refusée le serait aussi par le secours
if (STATUTS_SURCHARGE.includes(reponse.status)) {
  console.error('[chat] Gemini surchargé, passage au modèle de secours', reponse.status);
  ({ reponse, donnees } = await appelerGemini(modeleSecours, cleApi, contenus));
}
```

4. Afficher une réponse sans risque

Une réponse d'IA reste une donnée externe. Elle est affichée avec `textContent` et non `innerHTML` : même si elle contenait du HTML, il s'afficherait comme du texte au lieu de s'exécuter. C'est la protection de base contre l'injection de code (XSS).

js/script.js

```
function ajouterMessage(zone, role, texte) {
  const bulle = document.createElement('div');
  bulle.className = `message message-${role}`;
  bulle.textContent = texte;
  zone.appendChild(bulle);
  zone.scrollTop = zone.scrollHeight;
  return bulle;
}
```

8. La sécurité

Le dépôt GitHub est **public** : tout ce qui y entre doit être considéré comme lu par tout le monde, y compris dans l'historique Git.

Les secrets

Variable	Contenu	Où elle vit
`GEMINI_API_KEY`	Clé de l'IA	Vercel uniquement
`RESEND_API_KEY`	Clé d'envoi des e-mails	Vercel uniquement
`CONTACT_DESTINATAIRE`	Adresse qui reçoit les messages	Vercel uniquement

Le fichier `.gitignore` exclut les fichiers `.env` et `.vercel/`, et un test cherche les motifs de clés dans les fichiers du site. Si une clé est publiée par erreur, la retirer du fichier actuel ne suffit pas : elle reste dans l'historique. Il faut la **révoquer** et en créer une nouvelle.

Les en-têtes de sécurité (vercel.json)

Ils sont envoyés avec chaque réponse et donnent des consignes au navigateur.

En-tête	Effet
`Strict-Transport-Security`	Le navigateur n'utilise plus que HTTPS pour ce site.
`Content-Security-Policy`	Liste les seules sources autorisées pour les scripts, styles, polices, images et cadres. Un script injecté depuis ailleurs est bloqué.
`X-Frame-Options: DENY`	Le site ne peut pas être intégré dans une page piégée.
`X-Content-Type-Options: nosniff`	Le navigateur ne devine pas le type d'un fichier.

Le lien avec les règles du sujet. Sans style ni script en ligne, la CSP peut rester stricte, sans `unsafe-inline`. Les animations passent par `element.style.setProperty()` en JavaScript, ce que la CSP autorise, contrairement à un attribut `style=""` dans le HTML.

Bootstrap et l'intégrité des fichiers

Bootstrap est chargé depuis un CDN avec un attribut `integrity` : le navigateur compare l'empreinte du fichier reçu et le refuse s'il a été modifié.

9. Les tests et les preuves

La suite compte 74 tests répartis en trois fichiers. Ils utilisent le lanceur intégré à Node.js, sans dépendance.

Fichier	Ce qu'il vérifie
`tests/site.test.mjs`	Exigences du sujet, images, liens, absence de style en ligne, absence de secrets, HTTPS
`tests/contact.test.mjs`	Formulaire : validation, protections, erreurs, limite de débit
`tests/chat.test.mjs`	Chatbot : conversion de l'historique, erreurs, limite de débit, modèle de secours

Simuler le service externe

Un test ne doit pas envoyer de vrai e-mail. On remplace donc `fetch` par une fausse version, qui répond ce qu'on veut et mémorise ses appels. On peut alors vérifier ce qui aurait été envoyé, ou que rien n'a été envoyé.

tests/contact.test.mjs

```
function simulerResend(statut = 200, corps = { id: 'id-factice' }) {
  globalThis.fetch = async (url, options) => {
    appels.push({ url, options });
    return new Response(JSON.stringify(corps), { status: statut });
  };
}
```

tests/contact.test.mjs

```
test('POST ignore tout destinataire fourni par le visiteur', async () => {
  simulerResend();
  await POST(requete({ ...valide, to: 'victime@example.test', destinataire: 'victime@example.test' }));
  assert.deepEqual(JSON.parse(appels[0].options.body).to, [DESTINATAIRE]);
});
```

Une suite verte ne prouve pas tout

Trois habitudes de ce projet valent la peine d'être retenues :

- **La garde de comptage.** Si un fichier de tests ne se charge plus, les tests restants peuvent passer et donner une fausse impression de réussite. `tools/run-tests.mjs` échoue si leur nombre passe sous le minimum. Cela a déjà servi ici : une erreur de syntaxe a fait tomber le total de 61 à 35, et la suite a échoué.
- **Le contrôle négatif.** Pour savoir si un test protège vraiment, on casse volontairement le code. Ici, retirer la vérification du consentement a fait échouer exactement le test attendu, puis on a vérifié la restauration du fichier par son empreinte.
- **La preuve de bout en bout.** Des tests simulés ne prouvent pas qu'un e-mail arrive. L'envoi a été prouvé en retrouvant le message dans la boîte Gmail, et non en se fiant à la réponse `200`.

tools/run-tests.mjs

```
const total = reussis + echoues;
console.log(`\nattendu>=${MINIMUM_ATTENDU}, obtenu=${total}, réussis=${reussis}, échoués=${echoues}`);
if (echoues > 0 || total < MINIMUM_ATTENDU) {
  console.log('RÉSULTAT : ÉCHEC');
  process.exitCode = 1;
} else {
  console.log('RÉSULTAT : SUCCÈS');
```

Avant chaque commit

```
npm test             # doit afficher RÉSULTAT : SUCCÈS
npm run validate     # aucune erreur attendue
npm run dev          # contrôle visuel à 375 px et 1280 px
```

10. Le déploiement

Le projet est hébergé sur Vercel et relié au dépôt GitHub : **chaque `git push` sur `main` déclenche un déploiement**. On peut aussi déployer directement avec la commande `vercel deploy --prod`. Il n'y a pas d'étape de compilation : les fichiers sont publiés tels quels, et chaque fichier de `api/` dont le nom ne commence pas par `_` devient une fonction serveur.

Les variables d'environnement

```
vercel env ls                                  # affiche les noms, jamais les valeurs utiles
vercel env add RESEND_API_KEY production       # la clé est saisie à l'invite
vercel deploy --prod                           # une variable modifiée exige un redéploiement
```

Ce qui ne doit pas être publié

`.gitignore` protège Git, et `.vercelignore` protège le déploiement : ce sont deux listes distinctes. Sans la seconde, le sujet du projet, pourtant absent du dépôt, s'est retrouvé en ligne (voir le chapitre 12).

.vercelignore

```
# Fichiers jamais publiés par un déploiement depuis la ligne de commande
Projet 1.pdf
.env
.env.*
.git
tests
tools
docs
AGENTS.md
```

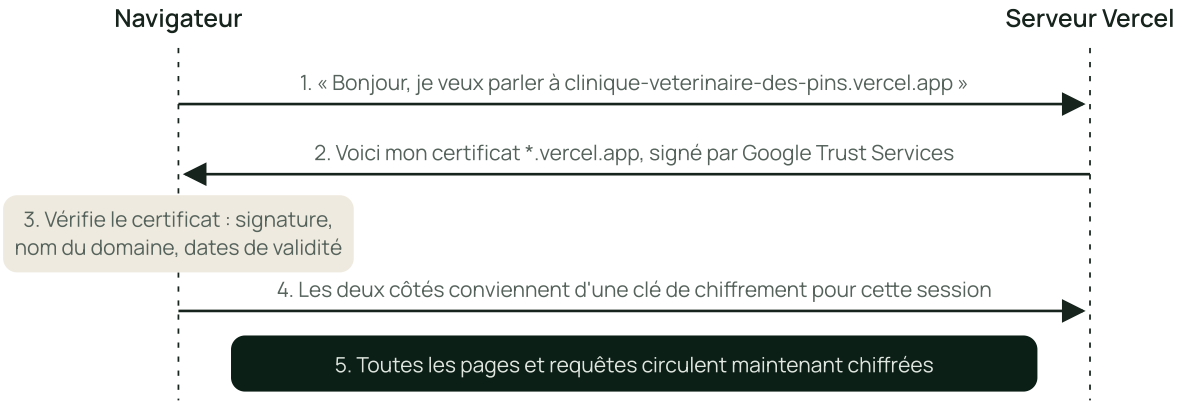
11. Comment nous avons obtenu HTTPS

Le site répond à l'adresse `https://clinique-veterinaire-des-pins.vercel.app`. Voici exactement ce qui a été fait, ce que Vercel a fait à notre place, et comment nous l'avons vérifié.

À quoi sert HTTPS

Avec HTTP, les échanges circulent en clair : n'importe quel intermédiaire, par exemple sur un Wi-Fi public, peut lire ou modifier ce qui passe. Ici, cela concernerait le nom, l'e-mail et le message saisis dans le formulaire. HTTPS, c'est HTTP protégé par **TLS**, qui apporte trois garanties :

Garantie	Ce que cela veut dire
Confidentialité	Les données sont chiffrées : un intermédiaire ne voit que du bruit.
Intégrité	Une modification en chemin est détectée.
Authenticité	Le certificat prouve que le serveur est bien celui de ce domaine.



Version simplifiée de l'établissement d'une connexion TLS. Tout se passe en quelques instants, avant le premier échange HTTP.

Ce qui a été fait, étape par étape

Étape	Qui	Comment
1. Héberger le site	Nous	`vercel link`, puis `vercel deploy --prod`. Le projet reçoit son adresse en `.vercel.app`.
2. Certificat TLS	Vercel, automatique	Aucune action : l'adresse est couverte par un certificat géré par Vercel.
3. Redirection de HTTP vers HTTPS	Vercel, automatique	Aucune action : toute visite en `http://` reçoit une redirection 308.
4. Imposer HTTPS au navigateur	Nous	En-tête HSTS ajouté dans `vercel.json`.
5. Aucune ressource en HTTP	Nous	Adresses relatives pour nos fichiers, `https://` pour les ressources externes, et un test qui surveille.

6. Prouver le résultat	Nous	Vérification sur le site en ligne, avec les commandes ci-dessous et dans Chrome.

Ce que nous n'avons pas eu à faire. Acheter un certificat, installer un serveur Nginx ou Apache, configurer Let's Encrypt, surveiller une date d'expiration, ou écrire du code de chiffrement. C'est l'un des avantages d'un hébergeur comme Vercel. Mais il faut comprendre ce qu'il fait pour vous, afin de savoir quoi vérifier.

Le certificat réellement utilisé

Relevé sur le site le 1er novembre 2026 :

Champ	Valeur	Explication
Sujet	`CN=*.vercel.app`	Certificat « joker » : il couvre tous les sous-domaines directs de `vercel.app`, dont le nôtre.
Émetteur	`Google Trust Services, WR1`	L'autorité qui se porte garante. Les navigateurs lui font confiance.
Valide du	31 octobre 2026	Environ 90 jours. Ces valeurs changeront : c'est Vercel qui gère le renouvellement, ne les recopiez pas dans votre code.
Valide jusqu'au	29 janvier 2027	

Notre part du travail : l'en-tête HSTS

La redirection a une faiblesse : la toute première requête part encore en HTTP, donc en clair. L'en-tête **HSTS** (*HTTP Strict Transport Security*) y répond : une fois reçu, le navigateur passe lui-même en HTTPS pour ce site, sans même essayer HTTP. Il suffit d'une ligne dans `vercel.json`, appliquée à toutes les pages par la règle `"source": "/(.*)"` :

vercel.json

```
"source": "/(.*)",
"headers": [
  { "key": "Strict-Transport-Security", "value": "max-age=31536000; includeSubDomains" },
```

- `max-age=31536000` : le navigateur retient la consigne pendant 31 536 000 secondes, soit un an.
- `includeSubDomains` : la règle vaut aussi pour les sous-domaines de cette adresse.

À manier avec prudence. Un navigateur qui a reçu HSTS s'en souvient jusqu'à la fin du `max-age`. Sur votre propre domaine, si HTTPS tombe en panne, vos visiteurs ne pourront plus passer par HTTP. Commencez par une courte durée, et n'activez `includeSubDomains` que si tous vos sous-domaines sont en HTTPS.

Éviter le « contenu mixte »

Une page en HTTPS qui charge un fichier en HTTP redevient vulnérable : c'est le **contenu mixte**, que les navigateurs bloquent ou signalent. Nos fichiers utilisent des adresses relatives (`css/style.css`), qui héritent automatiquement de HTTPS. Les ressources externes (Bootstrap, polices, carte) sont en `https://`. Un test échoue si une adresse en `http://` réapparaît, ou si l'en-tête HSTS disparaît :

tests/site.test.mjs

```
test('HTTPS : en-tête HSTS configuré et aucune ressource chargée en HTTP', async () => {
  const config = JSON.parse(await lire('vercel.json'));
  const entetes = config.headers.find((regle) => regle.source === '/(.*)').headers;
  const hsts = entetes.find((e) => e.key === 'Strict-Transport-Security');
  assert.ok(hsts, 'en-tête HSTS absent');
  assert.match(hsts.value, /max-age=31536000/);
  for (const fichier of [...PAGES, 'css/style.css', 'js/script.js', 'api/contact.js', 'api/chat.js']) {
    const contenu = await lire(fichier);
    assert.doesNotMatch(contenu, /http:\/\/(?!www\.w3\.org)/, `URL en HTTP dans ${fichier}`);
  }
});
```

Comment le vérifier vous-même

Ne dites jamais « c'est en HTTPS » sans l'avoir mesuré. Trois commandes suffisent :

```
# 1. HTTP doit rediriger vers HTTPS
curl -sI http://clinique-veterinaire-des-pins.vercel.app/

HTTP/1.0 308 Permanent Redirect
Location: https://clinique-veterinaire-des-pins.vercel.app/

# 2. HTTPS doit répondre et envoyer HSTS
curl -sI https://clinique-veterinaire-des-pins.vercel.app/

HTTP/1.1 200 OK
Strict-Transport-Security: max-age=31536000; includeSubDomains

# 3. Lire le certificat
echo | openssl s_client -connect clinique-veterinaire-des-pins.vercel.app:443 \
  -servername clinique-veterinaire-des-pins.vercel.app 2>/dev/null | openssl x509 -noout -issuer -subject
-dates

issuer=C=US, O=Google Trust Services, CN=WR1
subject=CN=*.vercel.app
notBefore=Oct 31 11:46:10 2026 GMT
notAfter=Jan 29 11:46:09 2027 GMT
```

Dans le navigateur, cliquez sur l'icône à gauche de l'adresse pour voir le certificat. L'onglet Réseau des outils de développement affiche les en-têtes, et la Console signale le contenu mixte. Sur ce site, le contrôle dans Chrome a compté **0 requête en HTTP** sur les 4 pages.

Et en local ?

`npm run dev` sert le site en `http://localhost:3000`, sans HTTPS. C'est normal : le trafic ne quitte pas votre machine, et les navigateurs traitent `localhost` comme un contexte sûr. `vercel.json` n'étant lu que par Vercel, HSTS et la CSP ne s'appliquent pas en local. Pensez donc à les vérifier une fois le site déployé.

Si vous n'êtes pas sur cette configuration

Ces cas n'ont **pas** été réalisés sur ce projet. Ils sont donnés pour vous orienter, à confirmer dans la documentation officielle :

- **Votre propre nom de domaine sur Vercel :** ajoutez-le au projet, configurez les enregistrements DNS indiqués, et Vercel émet le certificat.

- **Votre propre serveur :** il faut obtenir un certificat, par exemple gratuitement avec Let's Encrypt et l'outil `certbot`, configurer le serveur web, automatiser le renouvellement et mettre en place la redirection vous-même.

Ce que HTTPS ne fait pas

HTTPS protège le **trajet** des données, pas le reste. Il ne protège ni contre le spam, ni contre une faille du code, ni contre une clé oubliée dans le dépôt, et il ne garantit pas que le site soit honnête : un site malveillant peut très bien avoir un cadenas. C'est pourquoi il s'ajoute aux protections du chapitre 8, sans les remplacer.

12. Les erreurs réelles et leurs leçons

Ces problèmes sont vraiment arrivés pendant ce projet. Ce sont souvent les pages les plus utiles d'un guide : la méthode compte autant que la correction. À chaque fois : observer, mesurer, trouver la cause, corriger, puis vérifier.

Le contenu sautait au chargement

Symptôme. Un score de stabilité (CLS) de 0,29, pour un objectif sous 0,1.

Cause mesurée. L'API `PerformanceObserver` a désigné le coupable : le titre faisait 3 lignes avec la police de secours, puis 2 avec la vraie, plus étroite.

Correction. Une police de secours réduite aux mêmes proportions (`size-adjust`). Résultat en ligne : 0,003 au plus.

Leçon. Mesurez avant de corriger : l'intuition aurait accusé les images.

Un débordement horizontal de 12 px sur mobile

Cause. La classe Bootstrap `g-5` crée des marges négatives plus grandes que la marge du conteneur. Un script a listé les éléments qui dépassaient : `row g-5`, à droite jusqu'à 387 px sur un écran de 375.

Correction. `gy-5 gx-lg-5` : une grande gouttière horizontale seulement sur grand écran.

Leçon. Faites vérifier le débordement par un script : à l'œil, 12 px passent inaperçus.

Une animation qui fait déborder la page

Cause. Une photo agrandie à 114 % pendant son apparition, sans cadre en `overflow: hidden` pour la contenir.

Leçon. Un `transform` ne déplace pas ses voisins, mais il peut quand même créer une zone de défilement.

Des dimensions d'images fausses

Symptôme. Une photo déclarait 1800×1200 alors qu'elle mesurait 1800×3205. Plusieurs dizaines d'attributs étaient faux.

Correction. Attributs recalculés depuis les fichiers, et un test qui lit la taille réelle de chaque JPEG.

Leçon. Un bug corrigé sans test finit par revenir.

Un document privé publié

Cause. Le premier déploiement depuis la ligne de commande a envoyé des fichiers qui n'étaient pas dans Git, dont le PDF du sujet. Il a été accessible quelques minutes.

Correction. `.vercelignore`, redéploiement, suppression du déploiement fautif, puis vérification du code 404.

Leçon. Après un déploiement, vérifiez aussi ce qui *ne doit pas* être accessible.

Le serveur local exposait des secrets

Cause. Il écoutait sur toutes les interfaces réseau et servait tous les fichiers. `/.env.local` répondait 200 : quelqu'un sur le même Wi-Fi aurait pu le lire.

Correction. Écoute limitée à `127.0.0.1`, refus des fichiers cachés (réponse 403).

Leçon. Un outil de développement fait partie de la surface de sécurité.

Un chatbot qui échouait une fois sur trois

Symptôme. Des tests simulés tous verts, mais 4 appels réels sur 11 répondaient 502.

Cause mesurée. Les journaux de production indiquaient une surcharge de Gemini (503) et le quota gratuit (429). Notre propre mesure, trop rapide, avait dépassé ce quota.

Correction. Un modèle de secours. Le premier choisi répondait 404 en production malgré sa présence dans la liste des modèles : seul un appel réel l'a révélé. Après correction : 12 réponses sur 12, y compris en rafale.

Leçon. Un test simulé prouve votre logique, pas le service réel. Lisez les journaux, et testez en conditions réelles.

Une promesse non vérifiée

Cause. L'intégration Resend du Marketplace Vercel a été présentée comme gratuite avant vérification. Elle exigeait un domaine et coûtait au moins 20 $ par mois. Le compte Resend direct, lui, est gratuit.

Leçon. Vérifiez le prix et les conditions d'un service avant de le recommander, et corrigez ouvertement quand vous vous êtes trompé.

Un faux problème

Symptôme. Des sections vides sur des captures d'écran.

Cause. L'outil de capture défilait trop vite : avec un défilement réel, tous les éléments apparaissaient.

Leçon. Vérifiez votre instrument avant de corriger le code. Et quand une cause reste introuvable, écrivez « cause inconnue » plutôt qu'une explication inventée.

13. Recettes pour modifier le site

Changer une couleur ou une police

Modifiez la variable dans `:root`, en haut de `css/style.css`. Vérifiez ensuite le contraste du texte. Pour une police, changez aussi le lien Google Fonts dans les 4 pages, et recalculez `size-adjust` si elle a d'autres proportions.

Ajouter un service

1. Copiez une carte `col-md-6 col-lg-4` dans `services.html`.
2. Donnez à `data-categorie` l'une des valeurs existantes : `soins`, `specialites` ou `bien-etre`.
3. Complétez les rangées : un test exige un total de colonnes multiple de 12. Le test actuel attend aussi 12 cartes, à mettre à jour.
4. Ajoutez le service et son prix dans `api/_contexte.js`, sinon Sylva l'ignorera.

Ajouter une image

1. Ajoutez une ligne dans `tools/images.txt` : un test exige que chaque photo y soit listée.
2. Placez le fichier dans `assets/img/`, en le gardant léger.
3. Dans le HTML, indiquez `alt`, les `width` et `height` réels, et `loading="lazy"`.

Attention : `npm run images` retélécharge les photos d'origine, plus lourdes et d'autres dimensions. Les fichiers actuels ont été allégés après téléchargement. Après cette commande, il faut réaligner les attributs du HTML.

Modifier l'en-tête ou le pied de page

Faites la même modification dans les **4 fichiers HTML**, puis lancez `npm test` et `npm run validate`.

Changer une limite du formulaire

Les règles existent à deux endroits : les attributs HTML de `contact.html` pour le confort, et les constantes de `api/contact.js` pour la sécurité. Modifiez les deux et ajustez les tests.

Prendre une décision importante

Un nouveau service externe ou une nouvelle règle de sécurité se décrit d'abord dans un **ADR** (*Architecture Decision Record*) dans `docs/adr/` : le contexte, la décision, les options écartées et les conséquences. Dans six mois, c'est ce document qui expliquera pourquoi.

14. Exercices

Faites-les sur une branche à vous. Chaque exercice se vérifie avec `npm test`.

1. **Lecture.** Dans l'onglet Réseau de votre navigateur, soumettez le formulaire. Quelle requête part ? Pourquoi la clé n'y apparaît-elle pas ?
2. **Couleur.** Remplacez `--or` par une autre teinte, puis mesurez le contraste du bouton avec l'outil de votre navigateur.
3. **Animation.** Faites monter les mots du titre plus lentement. Quelle valeur faut-il changer, et dans quel fichier ? Activez ensuite « réduire les animations » dans votre système : que se passe-t-il ?
4. **Validation.** Ajoutez un champ « Nom de l'animal », facultatif, de 40 caractères au maximum. Où faut-il intervenir pour qu'il arrive dans l'e-mail ? Écrivez d'abord le test.
5. **Contrôle négatif.** Dans `api/contact.js`, désactivez le contrôle du champ piège. Quel test échoue ? Restaurez le fichier et prouvez-le avec `git diff`.
6. **Attaque.** Lisez `POST` dans `api/contact.js` : que se passe-t-il si une requête contient un champ `to` avec un autre destinataire ? Quel test le garantit ? Essayez ensuite avec `curl` sur votre serveur local : sans clé, il répond 503. Expliquez pourquoi ce contrôle passe en premier.
7. **Chatbot.** Ajoutez un service fictif dans `api/_contexte.js`, puis demandez son prix à Sylva. Demandez ensuite un service absent : invente-t-elle ?
8. **HTTPS.** Lancez les trois commandes du chapitre 11. Quelle est la date d'expiration actuelle du certificat ? Pourquoi une visite en `http://` reçoit-elle encore une réponse, et pourquoi votre navigateur ne l'envoie-t-il plus après une première visite ?
9. **Mesure.** Retirez temporairement les polices de secours du CSS et mesurez le CLS dans l'onglet Performance. Comparez avec le chapitre 12.

15. Limites connues et glossaire

Dettes assumées

Un projet réel a toujours des compromis. L'important est de les écrire, avec leur raison. La liste à jour est dans `docs/HANDOFF.md`.

Limite	Pourquoi on l'accepte ici
Limite de débit en mémoire, par instance	Suffisant pour un projet étudiant. Un trafic réel demanderait un stockage partagé.
En-tête et pied de page dupliqués	Le sujet impose du HTML statique, sans gabarit.
`style.css` de plus de 2 000 lignes	Le sujet impose un seul fichier. Il est découpé par un sommaire.
Clés collées un jour dans une conversation	Le propriétaire a décidé de les garder. À surveiller, et à remplacer en cas d'usage anormal. À ne pas reproduire.

Glossaire

ADR	Court document qui trace une décision d'architecture et ses raisons.
API	Interface qui permet à un programme d'en appeler un autre.
CDN	Réseau de serveurs qui distribue des fichiers près de l'utilisateur.
CLS	Score mesurant à quel point la page bouge pendant le chargement. Objectif : sous 0,1.
CSP	En-tête qui liste les sources autorisées et limite l'injection de code.
Fonction serveur	Petit programme qui s'exécute chez l'hébergeur à chaque requête, sans serveur à gérer.
HSTS	En-tête qui impose HTTPS au navigateur pour une durée donnée.
Parallaxe	Effet de profondeur : un élément se déplace à une autre vitesse que la page.
Variable d'environnement	Valeur fournie au programme au lancement, hors du code : l'endroit normal pour un secret.
XSS	Attaque qui fait exécuter du code malveillant dans la page d'un visiteur.